

1. Quelle contrainte REST impose l'absence de session ?

Réponse : Stateless.

2. Quelle méthode HTTP est sûre ET idempotente ?

Réponse : GET.

3. Quelle méthode HTTP est idempotente mais non sûre ?

Réponse : PUT et DELETE.

4. Pourquoi DELETE est idempotente ?

Réponse : Parce qu'exécuter plusieurs fois DELETE donne le même résultat final.

5. Quel code HTTP signifie « Conflict » ?

Réponse : 409 Conflict.

6. Quel code HTTP correspond à une mise à jour réussie sans retour ?

Réponse : 204 No Content.

7. Dans une API REST, la route doit être ?

Réponse : Claire, simple et basée sur les ressources.

8. Pourquoi MapControllerRoute() ne fonctionne pas correctement avec API REST ?

Réponse : Parce qu'il est destiné au MVC classique et non aux API REST.

9. Le contrôleur API hérite généralement de ?

Réponse : ControllerBase.

10. Pourquoi ControllerBase est utilisé ?

Réponse : Car il fournit les fonctionnalités nécessaires aux API sans les vues MVC.

11. Quelle annotation permet à OpenAPI de détecter un contrôleur ?

Réponse : [ApiController].

12. Quelle technologie génère automatiquement la documentation API ?

Réponse : OpenAPI / Swagger.

13. Quel package ajoute Scalar ?

Réponse : Scalar.AspNetCore.

14. Quelle erreur signifie problème de verbe HTTP ?

Réponse : 405 Method Not Allowed.

15. Quelle erreur signifie "Not Acceptable" ?

Réponse : 406 Not Acceptable.

16. POST n'est pas idempotent car ?

Réponse : Chaque appel peut créer une nouvelle ressource.

17. PATCH sert à ?

Réponse : Modifier partiellement une ressource.

18. Une erreur CORS est causée principalement par ?

Réponse : Le blocage des requêtes cross-origin par le navigateur.

19. CORS signifie ?

Réponse : Cross-Origin Resource Sharing.

20. Pourquoi POSTMAN fonctionne malgré CORS ?

Réponse : Parce que POSTMAN n'applique pas les règles CORS du navigateur.

21. AllowAnyOrigin() en production est ?

Réponse : Dangereux et déconseillé.

22. EF Core est ?

Réponse : Un ORM pour .NET.

23. ORM signifie ?

Réponse : Object Relational Mapping.

24. Quelle approche génère la base depuis les classes ?

Réponse : Code First.

25. Quelle approche est recommandée pour une BD existante ?

Réponse : Database First.

26. Une table EF correspond à ?

Réponse : Une classe Entity.

27. Quelle annotation change le nom de table ?

Réponse : [Table].

28. [StringLength] permet ?

Réponse : Définir une longueur minimale et maximale.

29. Une propriété de navigation permet ?

Réponse : Définir les relations entre entités.

30. ICollection dans EF sert souvent à ?

Réponse : Représenter une relation One-To-Many ou Many-To-Many.

31. Quelle annotation gère toute la ligne ?

Réponse : [Timestamp].

32. [ConcurrencyCheck] agit ?

Réponse : Sur des propriétés spécifiques pour gérer la concurrence.

33. LINQ signifie ?

Réponse : Language Integrated Query.

34. Quelle méthode évite InvalidOperationException ?

Réponse : FirstOrDefault() ou SingleOrDefault().

35. Quelle méthode supprime les doublons ?

Réponse : Distinct().

36. Quelle méthode permet le tri ?

Réponse : OrderBy().

37. ThenBy() sert à ?

Réponse : Ajouter un tri secondaire.

38. Skip(5) signifie ?

Réponse : Ignorer les 5 premiers éléments.

39. Take(5) signifie ?

Réponse : Prendre les 5 premiers éléments.

40. AsNoTracking() est utile pour ?

Réponse : Améliorer les performances en lecture seule.

41. Include() sert à ?

Réponse : Charger les relations liées.

42. Le mot-clé var ?

Réponse : Permet l'inférence automatique du type.

43. Les données API sont généralement transmises en ?

Réponse : JSON.

44. Quel code signifie "Forbidden" ?

Réponse : 403 Forbidden.

45. Quel code HTTP correspond à "Unauthorized" ?

Réponse : 401 Unauthorized.

46. DTO signifie principalement ?

Réponse : Data Transfer Object.

47. Le Mapper sert à ?

Réponse : Convertir Entity ↔ DTO.

48. Dans Add(), si le ModelState est invalide ?

Réponse : Retourner BadRequest(ModelState).

49. EF Core permet de travailler avec ?

Réponse : Des bases de données relationnelles via des objets .NET.

50. Quel attribut configure une clé composite récente ?

Réponse : [PrimaryKey].

51. Une propriété nullable FK est déclarée avec

Réponse : int? ou Guid?.

52. ToTable() appartient à :

Réponse : Fluent API.

53. LINQ-to-Entities génère :

Réponse : Du SQL.

54. Projection LINQ signifie :

Réponse : Sélection partielle des champs.

55. ToUpper() aide principalement à :

Réponse : Faire une recherche insensible à la casse.

56. Average() retourne :

Réponse : La moyenne.

57. SingleOrDefault() retourne :

Réponse : Un seul élément ou null.

58. Les requêtes SQL brutes sont utiles quand :

Réponse : LINQ ne suffit pas.

59. Encrypt=false dans chaîne connexion sert à

Réponse : Désactiver la vérification SSL.

60. Fluent API est utilisée quand :

Réponse : La configuration par annotations ne suffit pas.

Liste des annotations :

- Table** : Pour changer le nom de table
- Column** : Pour changer le nom de colonne
- Key** : Pour configurer la clé primaire
- ForeignKey** : Pour configurer la clé étrangère
- InverseProperty** : Pour configurer la relation ForeignKey
- Required** : Pour spécifier qu'un champs est obligatoire : not null
- NotMapped** : Pour spécifier qu'une classe ou champs ne doit pas être créée dans la BD
- ConcurrencyCheck** : Pour gérer la concurrence
- Timestamp** : Pour gérer la concurrence
- MaxLength** : Pour spécifier la longueur Max
- StringLength** : Pour spécifier la longueur Max et Min
- DatabaseGenerated** : Pour choisir l'algorithme de génération automatique
- ComplexType** : Pour spécifier qu'une classe doit être intégrée avec une autre dans la BD.

Résumé Rapide HTTP Methods

Méthode	Idempotente	Sûre
GET "Récupérer"	Oui	Oui
PUT "modifier"	Oui	Non
POST "ajouter"	Non	Non
DELETE	Oui	Non
PATCH modifier partiellement	Non	Non

CODE :

204 no content "put - patch - delete"

300 multiple choice

400 bad request

401 unauthorized

403 Forbidden = Vous êtes connecté, mais vous n'avez pas le droit d'accéder.

405 Method Not Allowed = Problème de verbe, vous venez avec un GET mais je suis un POST

406 Not Acceptable = Vous demandez du JSON, mais je suis programmé à fournir un array

408 Timeout = Timeout causé par le serveur cible

409 Conflict (POST - PUT) = Je comprend, C'est correcte mais ce n'est pas acceptable

504 No Response from Server= Time Out causé par un serveur utilisé par le serveur cible

Laquelle des contraintes REST interdit l'utilisation des sessions côté serveur ? : Stateless

Dans Program.cs, dans quel ordre faut-il configurer CORS ?

AddCors() avant Build(), UseCors() après Build() et avant UseAuthorization()

Combien de parties compose un JSON Web Token (JWT) ? :

Header.Payload.Signature

Quelle est la principale différence entre HS256 et RS256 ? :

HS256 utilise une clé symétrique (partagée), **RS256** utilise des clés asymétriques (privée/publique)

Où faut-il stocker la clé secrète JWT en production ? : Dans les variables d'environnement ou un gestionnaire de secrets (Azure Key Vault, AWS Secrets Manager)

Quelle commande remplace MapControllers() dans le Program.cs d'un projet YARP ? : MapReverseProxy

Comment YARP peut-il transmettre l'userid du token JWT aux microservices cibles ? : Via AddTransforms() qui extrait le claim NamelIdentifier et l'ajoute comme header X-User-Id
Dans une relation One-to-One, dans quelle table se trouve la clé étrangère ? Dans la table postérieure (créée en second dans l'ordre logique)

Pour une relation Many-to-Many PORTEUSE, quelle est la bonne approche ? Créer une classe intermédiaire + 2 relations One-to-Many
Que fait l'annotation [NotMapped] sur une propriété ? C)

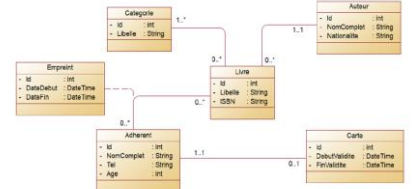
Exclut complètement la propriété de la génération de base de données
Comment récupérer l'étudiant le plus jeune avec LINQ ? db.Etudiants.OrderBy(e => e.Age).FirstOrDefault()

Comment vérifier si une filière a au moins un groupe avec LINQ ? db.Filieres.Where(f => f.Id == id).Any(f => f.Groupe.Any())

Comment travailler avec EFC ? :

- Microsoft.EntityFrameworkCore.Tools
- Microsoft.EntityFrameworkCore.SqlServer
- Code First** → Classes → Base de données
- Database First** → Base de données → Classes
- Model First** → Méthode ancienne/non recommandée

TP – Nous allons travailler sur ce diagramme de classes



```
using Microsoft.EntityFrameworkCore;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
```

namespace GestionBibliotheque

```
{
    // CATEGORIE
    public class Categorie
    {
        [Key]
        public int Id { get; set; }
        public string Libelle { get; set; }
        // Navigation
        public ICollection<Livre> Livres { get; set; }
    }

    // AUTEUR
    public class Auteur
    {
        [Key]
        public int Id { get; set; }
        public string NomComple { get; set; }
        public string Nationalite { get; set; }
        // Navigation
        public ICollection<Livre> Livres { get; set; }
    }

    // LIVRE
    public class Livre
    {
        [Key]
        public int Id { get; set; }
        public string Libelle { get; set; }
        public string ISBN { get; set; }
        // FK CATEGORIE
        public int CategorieId { get; set; }
        [ForeignKey("CategorieId")]
        public Categorie Categorie { get; set; }
        // FK AUTEUR
        public int AuteurId { get; set; }
        [ForeignKey("AuteurId")]
        public Auteur Auteur { get; set; }
        // MANY TO MANY PORTEUSE
        public ICollection<Emprunt> Emprunts { get; set; }
    }

    // ADHERENT
    public class Adherent
```

```

{
    [Key]
    public int Id { get; set; }
    public string NomComple { get; set; }
    public string Tel { get; set; }
    public int Age { get; set; }
    // ONE TO ONE
    public Carte Carte { get; set; }
    // MANY TO MANY PORTEUSE
    public ICollection<Emprient> Emprients { get; set; }
}
// CARTE
public class Carte
{
    [Key]
    public int Id { get; set; }
    public DateTime DebutValidite { get; set; }
    public DateTime FinValidite { get; set; }
    // ONE TO ONE
    public int AdherentId { get; set; }
    [ForeignKey("AdherentId")]
    public Adherent Adherent { get; set; }
}
// EMPRIENT
public class Emprient
{
    [Key]
    public int Id { get; set; }
    public DateTime DateDebut { get; set; }
    public DateTime DateFin { get; set; }
    // FK ADHERENT
    public int AdherentId { get; set; }
    [ForeignKey("AdherentId")]
    public Adherent Adherent { get; set; }
    // FK LIVRE
    public int LivreId { get; set; }
    [ForeignKey("LivreId")]
    public Livre Livre { get; set; }
}
// DB CONTEXT
public class MyContext : DbContext
{
    public MyContext(DbContextOptions<MyContext>
options)
        : base(options)
    {
    }
    public DbSet<Categorie> Categories { get; set; }
    public DbSet<Auteur> Auteurs { get; set; }
    public DbSet<Livre> Livres { get; set; }
    public DbSet<Adherent> Adherents { get; set; }
    public DbSet<Carte> Cartes { get; set; }
    public DbSet<Emprient> Emprients { get; set; }
}

```

Program.cs

```

builder.Services.AddDbContext<MyContext>(options =>
{
    options.UseSqlServer(
"Server=.;Database=BibliothequeDB;Trusted_Connection=
True;Encrypt=False");
});

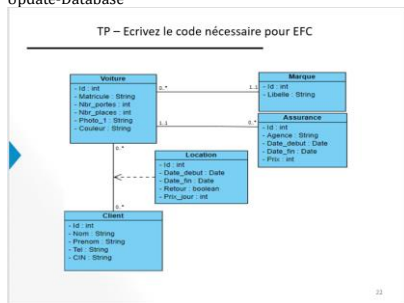
```

CMD MIGRATION

```

Add-Migration Creation
Update-Database

```



```

using Microsoft.EntityFrameworkCore;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
namespace LocationVoitureEFC
{

```

```

// MARQUE
public class Marque
{
    [Key]
    public int Id { get; set; }
    public string Libelle { get; set; }
    // Navigation
    public ICollection<Voiture> Voitures { get; set; }
}
// ASSURANCE
public class Assurance
{
    [Key]
    public int Id { get; set; }
    public string Agence { get; set; }
    public DateTime Date_debut { get; set; }
    public DateTime Date_fin { get; set; }
}

```

```

public int Prix { get; set; }
// Navigation
public ICollection<Voiture> Voitures { get; set; }
}
// VOITURE
public class Voiture
{
    [Key]
    public int Id { get; set; }
    public string Matricule { get; set; }
    public int Nbrportes { get; set; }
    public int Nbr_places { get; set; }
    public string Photo_1 { get; set; }
    public string Couleur { get; set; }
    // FK MARQUE
    public int MarqueId { get; set; }
    [ForeignKey("MarqueId")]
    public Marque Marque { get; set; }
    // FK ASSURANCE
    public int AssuranceId { get; set; }
    [ForeignKey("AssuranceId")]
    public Assurance Assurance { get; set; }
    // MANY TO MANY PORTEUSE
    public ICollection<Location> Locations { get; set; }
}
// CLIENT
public class Client
{
    [Key]
    public int Id { get; set; }
    public string Nom { get; set; }
    public string Prenom { get; set; }
    public string Tel { get; set; }
    public string CIN { get; set; }
    // Navigation
    public ICollection<Location> Locations { get; set; }
}
// LOCATION
public class Location
{
    [Key]
    public int Id { get; set; }
    public DateTime Date_debut { get; set; }
    public DateTime Date_fin { get; set; }
    public bool Retour { get; set; }
    public int Prix_jour { get; set; }
    // FK CLIENT
    public int ClientId { get; set; }
    [ForeignKey("ClientId")]
    public Client Client { get; set; }
    // FK VOITURE
    public int VoitureId { get; set; }
    [ForeignKey("VoitureId")]
    public Voiture Voiture { get; set; }
}
// DB CONTEXT
public class MyContext : DbContext
{
    public MyContext(DbContextOptions<MyContext>
options)
        : base(options)
    {
    }
    public DbSet<Voiture> Voitures { get; set; }
    public DbSet<Marque> Marques { get; set; }
    public DbSet<Assurance> Assurances { get; set; }
    public DbSet<Client> Clients { get; set; }
    public DbSet<Location> Locations { get; set; }
}

```

```

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddDbContext<MyContext>(options =>
{
    options.UseSqlServer(
"Server=.;Database=LocationVoitureDB;Trusted_Connection=
n=True;Encrypt=False");
});
// Commandes Migration
Add-Migration Creation
Update-Database

```

Configuration Program.cs

```

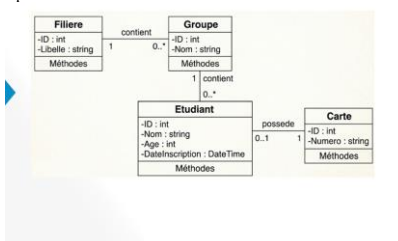
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddDbContext<MyContext>(options =>
{
    options.UseSqlServer(
"Server=.;Database=LocationVoitureDB;Trusted_Connection=
n=True;Encrypt=False");
});
// Commandes Migration
Add-Migration Creation
Update-Database

```

```

Add-Migration Creation
Update-Database

```



```

using Microsoft.EntityFrameworkCore;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
namespace GestionEcole
{

```

```

// FILIERE
public class Filiere
{

```

```

[Key]
public int ID { get; set; }
public string Libelle { get; set; }
// Navigation
public ICollection<Groupe> Groupes { get; set; }
}
// GROUPE
public class Groupe
{
    [Key]
    public int ID { get; set; }
    public string Nom { get; set; }
    // FOREIGN KEY FILIERE
    public int FiliereId { get; set; }
    [ForeignKey("FiliereId")]
    public Filiere Filiere { get; set; }
    // Navigation
    public ICollection<Etudiant> Etudiants { get; set; }
}
// ETUDIANT
public class Etudiant
{
    [Key]
    public int ID { get; set; }
    public string Nom { get; set; }
    public int Age { get; set; }
    public DateTime DateInscription { get; set; }
    // FOREIGN KEY GROUPE
    public int GroupId { get; set; }
    [ForeignKey("GroupId")]
    public Groupe Groupe { get; set; }
    // ONE TO ONE
    public Carte Carte { get; set; }
}
// CARTE
public class Carte
{
    [Key]
    public int ID { get; set; }
    public string Numero { get; set; }
    // FOREIGN KEY ETUDIANT
    public int EtudiantId { get; set; }
    [ForeignKey("EtudiantId")]
    public Etudiant Etudiant { get; set; }
}
// DB CONTEXT
public class MyContext : DbContext
{
    public MyContext(DbContextOptions<MyContext>
options)
        : base(options)
    {
    }
    public DbSet<Filiere> Filieres { get; set; }
    public DbSet<Groupe> Groupes { get; set; }
    public DbSet<Etudiant> Etudiants { get; set; }
    public DbSet<Carte> Cartes { get; set; }
}

```

Program.cs

```

using GestionEcole;
using Microsoft.EntityFrameworkCore;
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddDbContext<MyContext>(options =>
{
    options.UseSqlServer(
"Server=.;Database=GestionEcoleDB;Trusted_Connection=
True;Encrypt=False");
});

```

```

var app = builder.Build();
app.Run();

```

appsettings.json

```

{
    "ConnectionStrings": {
        "DefaultConnection":
"Server=.;Database=GestionEcoleDB;Trusted_Connection=
True;Encrypt=False"
    }
}

```

Commandes Migration

```

Add-Migration Creation
Update-Database

```